

1. Basic PHP Syntax

- PHP is a server-side scripting language, meaning the code runs on the server
- A PHP script can be placed anywhere in an HTML document.
- PHP code always starts and ends with special tags
- The default file extension for PHP files is .php.

When a user requests a PHP page:

- The request is sent to the web server.
- The server executes the PHP code.
- The output (usually HTML) is sent back to the browser.

Syntax:

```
<?php  
// PHP code goes here  
?>
```

1.1 What is PHP?

- PHP is a server-side scripting language used to build dynamic web pages and applications.
- It can interact with databases, handle forms, manage sessions, and generate dynamic content.
- PHP scripts are usually saved with the .php extension and executed by a web server such as Apache

Example:

```
<?php  
echo "Hello, World!";  
?>
```

Explanation:

- echo is used to display output.
 - When executed, it shows **Hello, World!** in the browser.
-

1.2 Statements in PHP

- A statement in PHP is any expression that is followed by a semicolon (;).
- Any sequence of valid PHP statements that is enclosed by the PHP tags is a valid PHP program.
- Here is a typical statement in PHP, which in this case assigns a string of characters to a variable called "\$greeting" –

```
$greeting = "Welcome to PHP!";
```

Example:

```
<?php
$greeting = "Hello";
$name = "Alice";

echo $greeting . ", " . $name . "!";
?>
```

Note:

- Missing a semicolon will cause a **syntax error**.
-

1.3 Embedding PHP in HTML

- One of PHP's strengths is its ability to be easily embedded within HTML.
- You can mix PHP code and HTML markup in the same file.
- This allows developers to create dynamic pages that include both static content (HTML) and dynamic content (PHP output).

Example:

```
<!DOCTYPE html>
<html>
<body>
<h1>Welcome!</h1>
<p>Today's date is:
<?php echo date("Y-m-d"); ?>
</p>
</body>
</html>
```

Here, the HTML structure remains static, while PHP dynamically generates the date.

1.4 PHP Tags and Structure

All PHP code must be written inside **PHP tags**, which tell the server where PHP code begins and ends.

Common tag styles:

```
<?php ... ?> // Standard and recommended  
<?= ... ?> // Short echo tag (outputs content)
```

Anything outside these tags is treated as plain HTML. PHP scripts are executed from top to bottom, just like other programming languages.

1.5 Comments in PHP

Comments are used to explain code and are ignored during execution. PHP supports single-line and multi-line comments:

Examples:

```
<?php  
// This is a single-line comment  
  
# This is also a single-line comment  
  
/*  
This is a multi-line comment.  
It can span several lines.  
*/  
?>
```

2. PHP Variables

- Variables in PHP are used to store data that can be accessed and modified across the program.
 - A variable can store a wide range of values, like numbers, text, arrays and even objects.
 - One of PHP's unique features is that it is a loosely typed language, which means you are not required to declare the data type of a variable when you create it.
 - PHP defines the variable's type based on the value assigned to it.
-

2.1 How to Declare a Variable in PHP?

To declare a variable in PHP, just assign a value by typing the \$ symbol followed by the variable name. PHP variables are case-sensitive and should begin with a letter or an underscore, followed by any number of letters, numbers or underscores.

Syntax:

```
$variable_name = value;
```

Example:

Here is an example showing how to declare variables in PHP –

```
// A string value
```

```
$name = "John";
```

```
// A number (integer)
```

```
$age = 25;
```

```
// A decimal number (float)
```

```
$price = 12.50;
```

2.2 PHP Variable Rules

Here is the list of rules to define a variable in PHP –

- A variable must start with a \$ symbol, then its name.

- The variable name must start with a letter or underscore (_).
- A variable name cannot start with a number.
- The variable name can contain letters, digits or underscores.
- PHP is case sensitive, so \$Name and \$name are distinct variables.

Example:

See the below example showing how both the variables \$Name and \$name are different to each other and see the output of the code.

```
$Name = "Amit";  
$name = "Samay";  
echo $Name;  
echo $name;
```

Output:

```
Amit  
Samay
```

2.3 Output Variables

The PHP [echo](#) keyword is often used to output data to the screen.

Example:

```
<!DOCTYPE html>  
<html>  
<body>  
<?php  
$txt1 = "Learn PHP";  
$txt2 = "W3Schools.com";  
echo "<h2>$txt1</h2>";  
echo "<p>Study PHP at $txt2</p>";  
?>
```

```
</body>
```

```
</html>
```

Output:

Learn PHP

Study PHP at [W3Schools.com](https://www.w3schools.com)

3. Data Types in PHP

- Every variable in PHP stores data of a certain **type**, which determines what kind of values it can hold and what operations can be performed on it.
 - PHP is a **loosely typed** language — this means you don't have to declare a variable's type explicitly. PHP automatically converts data types as needed during execution.
 - Understanding PHP data types is crucial for writing reliable, predictable, and efficient programs.
-

3.1 List of Data Types

PHP has a total of eight data types that we use to construct our variables –

- [Integers](#) – Whole numbers, without a decimal point, like 4195.
- [Doubles](#) – Floating-point numbers like 3.14159 or 49.1.
- [Booleans](#) – Have only two possible values, either true or false.
- [NULL](#) – Special type that only has one value: NULL.
- [Strings](#) – Sequences of characters, like 'PHP supports string operations.'
- [Arrays](#) – Named and indexed collections of other values.
- [Objects](#) – Instances of programmer-defined classes, which can package up both other kinds of values and functions that are specific to the class.

The first five are simple types, and the next two (arrays and objects) are compound types.

1. String

A string is a sequence of characters enclosed in quotes. You can use either single quotes (') or double quotes (").

Example:

```
<?php
$name = "Alice";
echo 'Hello, ' . $name; // Concatenation
echo "Hello, $name"; // Variable interpolation
?>
```

Output:

Hello, Alice

Hello, Alice

Difference between single and double quotes:

- Single quotes display text as-is.
 - Double quotes parse variables and escape sequences (like \n for a new line).
-

2. Integer

An integer is a whole number (positive, negative, or zero) without a decimal point.

Example:

```
<?php
$age = 25;
$year = -2025;
echo $age + 5;
?>
```

Output:

30

PHP automatically treats numeric values without decimals as integers.

3. Float (Double)

A float (also called a double) is a number with a decimal point or in exponential form.

Example:

```
<?php
$price = 100.50;
$tax = 0.10;
$total = $price + ($price * $tax);
echo $total;
?>
```

Output:

110.55

Floating-point numbers are used in calculations involving fractions or real-world measurements (like prices, percentages, etc.).

4. Boolean

A boolean represents only two values: TRUE or FALSE.

Example:

```
<?php
$isLoggedIn = true;
if ($isLoggedIn) {
    echo "Welcome back!";
} else {
    echo "Please log in.";
}
?>
```

Output:

Welcome back!

Booleans are commonly used in conditional statements to control program flow.

5. Array

An array is a collection of values stored under a single variable name.

Each value is associated with an index (numeric or associative).

Example (Indexed Array):

```
<?php
$colors = ["red", "green", "blue"];
echo $colors[1];
?>
```

Output:

green

Example (Associative Array):

```
<?php
$person = [
    "name" => "Alice",
    "age" => 25,
```

```
"city" => "London"
];
echo $person["city"];
?>
```

Output:

London

Arrays can hold any data type — even other arrays or objects — making them very flexible.

6. Object

An object is an instance of a class. Objects are used in object-oriented programming (OOP) to group data (properties) and behavior (methods) together.

Example:

```
<?php
class Car {
    public $brand;
    public $color;
    function __construct($brand, $color) {
        $this->brand = $brand;
        $this->color = $color;
    }

    function display() {
        echo "This car is a $this->color $this->brand.";
    }
}

$myCar = new Car("Toyota", "Red");
$myCar->display();
?>
```

Output:

This car is a Red Toyota.

Objects make PHP powerful for building complex applications.

7. NULL

The NULL data type represents a variable with no value.

A variable becomes NULL if it has been assigned the value NULL, has not been set, or has been unset.

Example:

```
<?php
$var = null;
if (is_null($var)) {
    echo "The variable is null.";
}
?>
```

Output:

Variable is NULL